سؤالات عملی (۱۰۰ نمره)

سؤال ۱ **[۶۰ نمره]**

در این تمرین عملی، شما یک سیستم هوشمند برای حل مسئله‌ی «برنامه‌ریزی و بهینه‌سازی سفر شهری» بر مبنای الگوریتم‌های Local Search طراحی و پیاده‌سازی خواهید کرد.
برخلاف روش‌های جستجوی گام‌به‌گام و کلاسیک مسیر (نظیر A* یا Dijkstra) که مسیر را رأس‌به‌رأس گسترش می‌دهند، در جستجوی محلی مسئله به صورت «پیکربندی کامل» (Complete-State Configuration) مدل‌سازی می‌شود. در دنیای واقعی، زمان سفر تنها تابعی از توپولوژی معابر نیست؛ بلکه شدیداً به «ساعت حرکت» وابسته است (برای مثال حرکت در ساعت 07:15 ترافیک کاملاً متفاوتی نسبت به ساعت 08:00 ایجاد می‌کند).

تعریف وضعیت و تابع هدف:
هر وضعیت در فضای حالت به صورت یک تاپل دوتایی تعریف می‌شود:

$$x = \langle P,\ t_{\text{start}} \rangle$$

- $P = \langle v_0, v_1, \ldots, v_m \rangle$ یک مسیر پیوسته و بدون دور (loop-free) از مبدأ ($v_0 =$ Home) تا مقصد ($v_m =$ Uni) روی گراف جهت‌دار شبکه معابر است.
- t_{start} ساعت حرکت از مبدأ در بازه‌ی زمانی مجاز روزانه است:

$$\mathcal{T} = \{ t_{\min} + k \cdot \Delta t \mid k \in \mathbf{N}_0,\ t_{\min} + k \cdot \Delta t \leq t_{\max} \}$$

زمان عبور از هر یال $e = (u, v)$ وابسته به زمان دقیق ورود خودرو به آن یال است:

$$\tau(e, t) = t_e^{\text{base}} \times \big(1 + C_e(t)\big)$$

که در آن $C_e(t)$ ضریب ترافیک متغیر با زمان بر مبنای امواج گوسی ساعت اوج شلوغی است.
هدف، یافتن وضعیتی است که تابع هزینه سفر $f(x)$ را کمینه کند (محاسبه در شبیه‌ساز FIFO با پیش‌روی زمان). وضعیت‌های دارای دور، گسستگی در مسیر یا ساعت حرکت خارج از بازه، مشمول جریمه بی‌نهایت ($f(x) = \infty$) خواهند شد.

بخش‌های پیاده‌سازی (پوشه /algorithms):
شما باید فایل‌های موجود در پوشه /algorithms را بدون تغییر ساختار امضای توابع پیاده‌سازی نمایید:

۱. **عملگرهای همسایگی و ترکیب (operators.py):**

- تابع get_neighbors: تولید همسایگان معتبر با اعمال جهش روی ساعت حرکت ($\pm\Delta t$) و تغییر موضعی زیرمسیرها (ایجاد detour بدون دور).
- تابع crossover: ترکیب ساختار دو مسیر والد بر مبنای تقاطع‌های مشترک بینابینی و میانگین‌گیری از ساعت حرکت.

۲. **الگوریتم Hill Climbing (hill_climbing.py):** پیاده‌سازی Steepest Descent با پشتیبانی از حرکات افقی روی دشت‌ها (Sideways Moves) تا سقف مشخص و بازنشانی‌های تصادفی (Random Restarts) برای خروج از بهینه‌های محلی.

۳. **الگوریتم Simulated Annealing (simulated_annealing.py):** پیاده‌سازی برنامه زمان‌بندی افت دما ($T_{k+1} = \alpha T_k$) و پذیرش احتمالی وضعیت‌های بدتر طبق معیار Metropolis ($P = \exp(-\Delta E / T)$).

۴. **الگوریتم Local Beam Search (beam_search.py):** نگهداری هم‌زمان K وضعیت موازی، تجمیع همه‌ی همسایگان حاصل از این K وضعیت و انتخاب مجدد K وضعیت برتر از میان کل کاندیداها.

۵. **الگوریتم Genetic Algorithm (genetic_algorithm.py):** مدیریت جمعیت، تعریف تابع Fitness، انتخاب والدین (Tournament یا Roulette Wheel)، بازترکیب با crossover، جهش تصادفی با نرخ معین و حفظ نخبگان (Elitism).

مدیریت ابرپارامترها (config.json):
تمامی پارامترهای تنظیمی الگوریتم‌ها در فایل config.json متمرکز شده‌اند. شما می‌توانید این مقادیر را برای دستیابی به بهترین کارایی تغییر دهید یا متغیرهای جدید دلخواه خود را در آن تعریف کنید.

ابزارهای بصری‌سازی و اجرا:
برای مشاهده رفتار ترافیکی شبکه قبل از شروع کدنویسی، می‌توانید از ابزار تعاملی زیر استفاده نمایید:

```
python main.py --interactive
```

همچنین برای اجرای الگوریتم‌ها، رسم مسیر بهینه و رسم نمودار همگرایی ($f(x)$ بر حسب تکرار)، از آرگومان‌های plot-route– و plot-metrics– در فایل main.py استفاده کنید.

گزارش کار تحلیلی (REPORT.md یا PDF):
همراه با کدها، ارائه‌ی یک گزارش الزامی است. این گزارش می‌تواند به **زبان فارسی یا انگلیسی** نوشته شود (انتخاب با خود دانشجو است).
نکته بسیار مهم: انتظار متن‌های طولانی و کلیشه‌ای نداریم؛ پاسخ‌های دقیق، متفکرانه و مختصر مد نظر است. نگارش گزارش **اکیداً باید توسط خود دانشجو** انجام شود و استفاده از ابزارهای LLM برای تولید متن گزارش ممنوع است.

موارد الزامی در گزارش:

- شرح منطق پیاده‌سازی عملگرهای همسایگی و چگونگی تضمین بدون دور ماندن مسیرها.
- جدول مقایسه‌ای عملکرد هر 4 الگوریتم (بهترین زمان سفر $f(x^*)$، ساعت حرکت بهینه، زمان اجرای برنامه و تعداد گام تا همگرایی).
- قرار دادن تصویر نمودار همگرایی (convergence_comparison.png) و تحلیل مقایسه‌ای رفتار الگوریتم‌ها روی دشت‌ها، قله‌ها و دره‌های محلی.
- **پاسخ به حداقل 5 پرسش از 7 پرسش تحلیلی زیر** (پاسخ‌های شما بررسی شده و بهترین 5 پاسخ نمره‌دهی خواهند شد):

۱. **پرسش اول (Continuous Optimization & Gradient Descent):** در مسائل بهینه‌سازی پیوسته، الگوریتم Gradient Descent متغیرها را با رابطه‌ی $x \leftarrow x - \alpha \nabla f(x)$ به‌روزرسانی می‌کند.

- چرا الگوریتم Gradient Descent کلاسیک اساساً برای بهینه‌سازی مؤلفه مسیر (P) در این گراف معابر قابل استفاده نیست؟
- آیا می‌توان از Gradient Descent صرفاً برای بهینه‌سازی ساعت حرکت t_{start} (با فرض ثابت بودن مسیر P) استفاده کرد؟ تابع ترافیک پویا ($C_e(t)$) چه شرایط ریاضی مشخصی را باید ارضا کند تا چنین کاری مجاز باشد؟

۲. **پرسش دوم (Local Search vs. Classical Incremental Search):** الگوریتم‌های کلاسیک جستجوی گراف (مانند A* یا Dijkstra) مسیر را به صورت گام‌به‌گام می‌سازند.

- در این مسئله، چرا برنامه‌ریزی سفر به صورت یک پیکربندی کامل ($x = \langle P, t_{\text{start}} \rangle$) مدل شده است و نه جستجوی فضای حالت تدریجی؟
- تحت چه شرایط یا محدودیت‌هایی در شبکه‌های شهری، الگوریتم‌های تدریجی مانند Time-Dependent Dijkstra در مقایسه با روش‌های Local Search دچار چالش یا ناکارآمدی می‌شوند؟

۳. **پرسش سوم (Local Beam Search vs. Genetic Algorithm):** در اسلایدهای درس، الگوریتم Local Beam Search به عنوان نگهداری K جستجوی موازی معرفی شده و به شباهت آن با ایده‌ی تکامل زیستی (Evolution) اشاره می‌شود.

- عملگر Crossover در الگوریتم ژنتیک چه مزیت بنیادینی نسبت به مکانیزم انتخاب حریصانه‌ی ادغام‌شده در Local Beam Search ایجاد می‌کند؟
- آیا الگوریتم Local Beam Search شما دچار مشکل کلاسیک همگرا شدن هر K پرتو به یک کمینه محلی تکراری (Population Collapse) شد؟ چگونه می‌توان تنوع پرتوها را حفظ کرد؟

۴. **پرسش چهارم (Plateaus & Sideways Moves):** در مسئله‌ی 8-Queens، دادن مجوز تا سقف 100 حرکت افقی (Sideways Moves)، شانس موفقیت تپه‌نوردی را از 14% به 94% رساند.

- در این شبکه معابر، چه سناریوی واقعی باعث ایجاد دشت یا وضعیت‌های مسطح (Shoulder / Flat Local Extrema) می‌شود که در آن همسایگان زمان سفر یکسان یا تقریباً یکسانی دارند؟
- تنظیم مقدار ابرپارامتر max_sideways چه تأثیری روی زمان اجرای الگوریتم در برابر کیفیت جواب نهایی داشت؟

۵. **پرسش پنجم (Cooling Schedule Dynamics):** الگوریتم Simulated Annealing بر پایه‌ی احتمال پذیرش متروپولیس $P = \exp(-\Delta E/T)$ عمل می‌کند.

- ریسک‌های انتخاب دمای اولیه‌ی (T_0) بسیار بالا یا بسیار پایین چیست؟
- اگر نرخ خنک‌سازی (α) خیلی کوچک باشد (مانند $\alpha = ۰/۵$ یا سردسازی ناگهانی/Quenching) در مقایسه با حالتی که بسیار نزدیک به یک باشد (مانند $\alpha = ۰/۹۹۹$)، چه تغییری در رفتار جستجو مشاهده کردید؟

۶. **پرسش ششم (Crossover Disjointness & Epistasis):** دو جواب والد $x_1 = (P_1, t_1)$ و $x_2 = (P_2, t_2)$ را در نظر بگیرید که از دو مسیر کاملاً مجزا از شهر عبور می‌کنند و هیچ گره مشترکی به جز مبدأ و مقصد ندارند.

- تابع crossover شما چگونه با این حالت برخورد می‌کند تا خروجی نامعتبر یا دارای قطع‌شدگی تولید نشود؟
- چرا ترکیب دو مسیر والد باکیفیت ممکن است در مواردی فرزندی با کیفیت بسیار ضعیف و نامطلوب تولید کند (High Epistasis)؟

۷. **پرسش هفتم (Combinatorial State Space):** فرض کنید گراف دارای $|V|$ گره، میانگین درجه خروجی b برای هر گره، میانگین طول مسیر L گام (یال) از مبدأ تا مقصد، و بازه زمانی حرکت با N_t گسست زمانی باشد.

- یک کران بالای تئوری برای تعداد کل وضعیت‌های فضای حالت ($|\mathcal{X}|$) فرمول‌بندی کنید.
- با توجه به این فرمول، توضیح دهید چرا جستجوی کامل و جامع (Brute-force) برای شبکه‌های واقعی غیرممکن است و استفاده از الگوریتم‌های Local Search ضرورت پیدا می‌کند.

نحوه تحویل: شما باید این مخزن را در حساب گیت‌هاب شخصی خود Fork کرده و با کامیت‌های منظم پیشرفت کار خود را ثبت نمایید. در نهایت لینک مخزن خود را در سامانه تحویل ارسال کنید.
توجه: تمامی تغییرات باید قبل از ددلاین کامیت و پوش شده باشند. هرگونه تغییر یا کامیت پس از ددلاین مشمول جریمه سنگین یا نمره صفر در آن بخش خواهد شد.

لینک مخزن کدهای اولیه برای fork کردن: https://github.com/AliMozdian/commute-optimizer

سؤال ۲ **[۴۰ نمره]**

این بازی مشابه بازی Pacman می باید با این تفاوت که نقطه ای برای امتیاز گرفتن وجود ندارد و تنها راه امتیاز گرفتن در این بازی به قتل رساندن روح هاست.

در هر استیت از بازی ۲ اکشن مجاز وجود دارد: حرکت به یکی از tile های همسایه یا کشتن یکی از روح هایی که در tile های همسایه. کشتن یک روح در tile همسایه موجب رفتن کاراکتر به آن tile می شود.

در ابتدای بازی یک تیر دریافت می کنید و به ازای پیمایش هر ۱۰ خانه یک تیر جدید به شما داده می شود.
محیط بازی هر بار به صورت رندم با تعدادی دیوار تولید می شود.
هدف ما زنده ماندن و کشتن بیشترین تعداد روح هاست.

برای این تمرین الگوریتم های minimax ، alphabeta-pruning و Montecarlo-Treesearch را باید به کمک TODO های موجود در فایل داده شده پیاده سازی کنید.

بازی بعد از ۱۰۰ حرکت به طور اتوماتیک برای جلوگیری از حالت های stale متوقف می شود.

در الگوریتم های Alphabeta و minimax ، عمق جستجو دست خود شماست
در الگوریتم MonteCarlo تعداد iteration ها و rollout ها و همچنین rollout policy دست خود شماست.
زمان پردازش برای هر حرکت نمی تواند بیشتر از ۵ ثانیه باشد و در صورتی که از محدودیت زمانی عبور کنید حرکتی انجام نخواهید داد.